

DOCUMENTAÇÃO DE ENGENHARIA

DOCUMENTAÇÃO TÉCNICA — PORTAL GUNDAM TCG BR

Referência para quem vai manter e estender o sistema: arquitetura, modelo de dados, pipeline de catálogo, tradução pt-BR, motor do simulador, rede em tempo real e ferramentas de desenvolvimento.

Versão do documento: 1.0 · App: v1.0.0

Branch de referência: dev · Data: 2026-09-06

Base: código do repositório + docs/ internos + CHANGELOG.md

PORTAL-GUNDAM-TCG-BR · DOCUMENTAÇÃO INTERNA

Sumário

1. Visão geral e stack
2. Como rodar local
3. Modelo de dados
4. Catálogo de cartas
5. Tradução pt-BR do texto de efeito
6. Motor do simulador
7. Simulador — servidor autoritativo e rede
8. Deckbuilder e legalidade de deck
9. Ferramentas de desenvolvimento
10. Convenções
11. Limitações conhecidas e backlog

1. Visão geral e stack

O Portal Gundam TCG BR é uma aplicação web única (monorepo simples, sem workspaces de pacote separados) que reúne quatro produtos: catálogo de cartas, deckbuilder, base de regras (rulings) e simulador de partidas. O frontend é uma SPA React servida como estático; o backend é uma API Express + Prisma sobre PostgreSQL; o simulador acrescenta um motor de regras puro em TypeScript e uma camada de rede em tempo real (SSE + Socket.io).

1.1 Stack

Camada	Tecnologia	Observações
Frontend	React 19, Vite 7, TypeScript 5.9, Tailwind CSS 4	Componentes Radix UI + shadcn (components.json), framer-motion , recharts (gráficos do deckbuilder/stats), @dnd-kit (arrastar-soltar nas Pastas).
Roteamento	wouter 3	Hash routing — src/App.tsx usa <Router hook={useHashLocationWithQuery}> (src/lib/hashLocationWithQuery.ts), que preserva query string no fragmento. Deploy estático não precisa de rewrite de servidor.
Estado global	React Context	src/contexts/ : AuthContext (JWT + usuário), ThemeContext (claro/escuro), FactionContext .
HTTP client	axios (src/lib/api.ts)	Conversão de caso camelCase ⇌ snake_case nas bordas (ver §10).
Backend	Express 5, Node 22+, tsx (runtime, sem build)	Arquivo único server/index.ts (~3.5k linhas). jsonwebtoken , bcryptjs , google-auth-library , multer (upload), cors .
ORM / banco	Prisma 6, PostgreSQL	prisma/schema.prisma . Banco local via Docker (docker-compose.yml).
Rede do simulador	socket.io 4 + SSE nativo	server/simulatorSocket.ts (Socket.io) roda ao lado das rotas SSE/POST. Cliente: socket.io-client .
Testes	Vitest 4, Testing Library, jsdom	pnpm test — cerca de 640 casos. Sem subir servidor: motor e libs são funções puras.
Gerenciador	pnpm 10 via corepack	packageManager fixado no package.json . Comandos: corepack pnpm
Ferramentas de IA/motor	MCP (@modelcontextprotocol/sdk)	scripts/mcp-gundam/ — servidor MCP que expõe motor + catálogo. Registrado em .mcp.json .

1.2 Layout de pastas

```
portal-gundam-tcg-br/
├─ server/
│  ├─ index.ts           API Express (todas as rotas /api/*)
│  └─ simulatorSocket.ts camada Socket.io do simulador
├─ src/
│  ├─ App.tsx, main.tsx  shell + router (hash)
│  ├─ pages/             uma página por rota (CardsPage, DeckbuilderPage,
│                        SimulatorSandboxPage, SimulatorMatchPage, AdminPage...)
│  ├─ components/        UI compartilhada (shadcn/Radix + componentes do domínio)
│  ├─ contexts/          AuthContext, ThemeContext, FactionContext
│  ├─ lib/               api.ts, deck-legality.ts, deck-level-stats.ts,
│                        gundam-card-effects.ts, gundam-catalog.ts, utils
│  └─ modules/
│     ├─ cards/ decks/ rules/ tournaments/ admin/  (domínios do portal)
│     └─ simulator/  (o motor – ver §6/§7)
│        ├─ engine/   motor puro (types, events, phases, combat, effectSpec, ...)
│        ├─ content/  EffectSpec por wave (st01..st04) + predicates
│        ├─ fixtures/ CardDef + decklists dos 4 starters + vanillaDeck
│        ├─ server/   matchStore.ts, socketBridge.ts (autoritativo, server-side)
│        ├─ network/  socketClient.ts, useMatchTransport.ts (cliente)
│        └─ ui/       componentes/hlevo visuais da partida
├─ prisma/             schema + migrations + seeds + scripts de curadoria
├─ scripts/            catalog-import, translate-card-effects, gundam-fuzz,
│                      mcp-gundam/, db-backup/restore, dev-api.mjs
├─ data/              datasets-fonte do catálogo (json)
├─ docs/              01..46 – decisões de arquitetura e execução
├─ tutoriais/         documentacao/ manuais e este documento
└─ docker-compose.yml render.yaml vercel.json .github/workflows/ci.yml
```

1.3 Deploy

Alvo	O quê	Config
Vercel	Frontend estático (build Vite)	vercel.json . Domínio público portal-gundam-tcg-br.vercel.app .
Render	API Node (<code>pnpm run start = prisma migrate deploy + tsx server/index.ts</code>)	render.yaml — <code>branch: main</code> , <code>healthcheck /api/health</code> , todas as secrets (<code>DATABASE_URL</code> , <code>JWT_SECRET</code> , <code>GOOGLE_CLIENT_ID</code> , chaves Supabase Storage...) preenchidas no painel, nunca commitadas.
Supabase	PostgreSQL gerenciado (produção) + Storage de imagens de carta	<code>STORAGE_DRIVER=supabase</code> no backend. Local usa <code>STORAGE_DRIVER=local</code> (grava em <code>public/uploads</code>).

O modelo de branch é **staging via dev** e **produção via main** (Render aponta pra `main`). Ver `docs/01-arquitetura-roadmap.md` , `docs/15-deploy-e-login-google.md` .

2. Como rodar local

2.1 Requisitos

- Node.js 22+ (24 ok), pnpm 10+ via corepack enable
- Docker Desktop / Engine + Compose (PostgreSQL local)

2.2 Do zero

```
git clone <url> && cd portal-gundam-tcg-br
corepack pnpm install
cp .env.example .env # ajustar se necessário (ver 2.4)
corepack pnpm run db:up # sobe o container postgres (docker compose)
corepack pnpm run setup:fresh-env # prisma generate + migrate deploy + catalog:bootstrap
corepack pnpm run dev:full # API (porta 8787) + frontend (Vite) juntos
```

dev:full = pnpm run dev:api (que roda scripts/dev-api.mjs → prisma generate + prisma db push + tsx server/index.ts) e pnpm run dev (Vite).

2.3 Bootstrap do catálogo

setup:fresh-env chama catalog:bootstrap, que é uma cadeia (ver package.json):

```
prisma:seed # usuário admin + registros de exemplo
prisma:seed:apitcg # importa o dataset APITCG (data/apitcg-gundam.json) → ~1.812 cartas / 22 sets
cardmodel:migrate:apply # sincroniza o CardModel (identidade de jogo) de cada Card (print)
curation:gcg:apply # curadoria oficial: série, sourceTitle, relações (data/gcg-official-cards.json)
keywords:extract:apply # extrai triggerKeywords/effectKeywords/hasBurst/... do texto
banlist:apply # aplica a banlist oficial (legalityStatus / CardBanGroup)
catalog:apitcg:check # valida o dataset importado
```

Variante destrutiva: catalog:bootstrap:fresh (= prisma db push --force-reset + catalog:bootstrap). Só catálogo, sem resetar o banco: catalog:bootstrap.

2.4 .env

Copiado de .env.example. Chaves relevantes:

Variável	Uso
DATABASE_URL / SHADOW_DATABASE_URL	Postgres local (Docker) e banco-sombra fixo pra prisma migrate dev.
API_PORT (8787)	Porta da API.
JWT_SECRET	Assinatura dos tokens (login e handshake do Socket.io).
SEED_ADMIN_EMAIL / SEED_ADMIN_PASSWORD	Conta admin criada pelo prisma:seed (default admin@gundambr.local / admin123).
ALLOWED_ORIGINS	Vazio local = CORS aberto. Em produção, domínios do front separados por vírgula.
GOOGLE_CLIENT_ID / VITE_GOOGLE_CLIENT_ID	Login Google (opcional — sem isso o botão some e a rota responde "não configurado").
STORAGE_DRIVER	local (grava em public/uploads, servido em /uploads) ou supabase.
VITE_API_BASE_URL	http://localhost:8787/api em dev.
VITE_LAYOUT_PREVIEW	1 libera /simulador/preview-layout sem login fora do modo DEV (staging).
GEMINI_API_KEY	Lido de .spartan/ai.env pelo pipeline de tradução (\$5).

Nunca commitar credenciais. `.env.example` traz alguns valores de exemplo/placeholder; trate todos como não-secrets e substitua no `.env` real. As secrets de produção vivem só no painel do Render / Supabase.

2.5 Contas seed

`prisma:seed` cria a conta admin (`SEED_ADMIN_*`) com `role = ADMIN` . Novos usuários entram como `role = USER` . Papel `EDITOR` e a flag `isHoster` são atribuídos pelo admin. Para navegar no admin sem catálogo completo, `prisma:seed` já basta.

2.6 Migrations

- Ambiente novo: `prisma migrate deploy` (aplica o histórico de `prisma/migrations/` sem banco-sombra).
- Dia a dia: `dev:api` roda `prisma db push` (sincroniza estrutura, não versiona).
- Mudança de schema versionada: `prisma:migrate --name <nome>` . Antes, ver `docs/11-checklist-migration.md` (risco de *drift* entre `db push` e `migrate`).
- Backup/restore do banco: `pnpm run db:backup` / `db:restore` (`scripts/db-*.mjs` , ver `docs/12`).

3. Modelo de dados

Fonte: `prisma/schema.prisma` . Regras de schema: `.cLaude/ruLes/database/SCHEMA.md` .

Postgres com Prisma. Convenções do projeto (ver §10 e `SCHEMA.md`): **sem foreign keys / sem CASCADE no banco** — as relações do Prisma existem no client mas a integridade é responsabilidade da aplicação; **soft delete** via `deletedAt` nos modelos de catálogo/evento; timestamps `createdAt` / `updatedAt` ; PKs `cuid()` .

Exceção prática: alguns modelos *de junção* puramente internos (`DeckItem` , `CardBinderItem` , `CardRelation` , tabelas de evento) declaram `onDelete: Cascade / Restrict / SetNull` no Prisma por conveniência de manutenção. O catálogo e as entidades de usuário seguem a regra "sem FK / soft delete".

3.1 Modelos principais

Modelo	Papel	Campos-chave
User	Conta. Login email/senha (passwordHash bcrypt) ou Google (googleId).	role (USER/EDITOR/ADMIN), isHoster , preferredCardLanguage (PT_BR/EN), username único.
Season	Temporada de metagame — agrupa um lançamento principal + produtos da mesma "era". Só uma com isCurrent .	code , isCurrent . Referenciada por CardSet , Tournament , HostedEvent .
CardSet	Coleção/produto (booster, starter deck, acessório).	code , setType (SetKind), seasonId? , starterDeckVariantOf? , metadataJson .
CardModel	Identidade de jogo da carta — nome, efeito, stats, traits, keywords. Compartilhado por todas as impressões do mesmo code .	code único, cardType (CardType), effectEn / effectPt , triggerKeywords[] / effectKeywords[] / keywordTags[] , hasBurst/hasMain/hasAction/oncePerTurn , textSectionsJson , legalityStatus , banGroupId? .
Card	Impressão (print) — uma tiragem física: raridade, imagem, set, código de produto. Aponta pro CardModel .	cardModelId? , setId? , rarity , isPrimaryPrint , printLabel , imageUrl / image{Small,Medium,Large}Url , externalId único. Muitos @@index pra filtros do catálogo.
CardBanGroup	Agrupa CardModel mutuamente incompatíveis além de maxDistinct escolhas — cobre par banido e a regra de categoria genérica oficial.	maxDistinct , members: CardModel[] .
CardRelation	Relação curada entre modelos (piloto de, apoia, upgrade, mesmo arquétipo, story).	sourceModelId , targetModelId , relationType (CardRelationType).
TaxonomyEntry	Vocabulário controlado: TRAIT ou SOURCE_TITLE (série/mídia). Com capa + link oficial pro admin.	kind , name , slug (únicos por kind).
Deck / DeckItem	Deck do usuário e suas cartas.	Deck: visibility (DeckVisibility), shareId único, coverImage , featuredCardIds[] , isPrimary . DeckItem: cardId , quantity , section (main / resource / ex_base / ex_resource / token_reference), único por (deckId, cardId, section) .
CardBinder / CardBinderItem	Pastas (coleção pessoal) e itens, com position pra ordenação drag-and-drop.	shareId único, isPublic .
Ruling	Regra/FAQ. Original EN + tradução PT + exemplo. Pode se vincular a Card ou CardModel .	sourceType (RuleSourceType), questionEn/PtBr , answerEn/Pt , relatedKeyword , relatedPhase , translationStatus .

Tournament / TournamentEntry	Report retroativo de torneio (cadastrado pelo admin). TournamentEntry aceita jogador convidado (sem conta).	seasonId? (relacional; season string é legado), placement , archetype , deckId? , userId? , deckSnapshotId? .
TournamentEntryDeckChangeLog	Auditoria de troca do deck vinculado a um resultado histórico (quem, quando, snapshot anterior).	—
HostedEvent + Participant / Round / Match	Evento "ao vivo" organizado por um isHoster /ADMIN. Fases: esqueleto → participantes (sempre conta cadastrada) → rodadas/confrontos (pareamento manual no MVP).	HostedEventStatus , HostedEventRoundStatus , HostedEventMatchResult (inclui BYE). Participant.deckSnapshotId trava a decklist — não pode reescrever depois.
DeckSnapshot / DeckSnapshotItem	Cópia congelada de uma decklist no momento em que foi vinculada a um resultado histórico ou travada num evento. Sobrevive à edição/remoção do Deck de origem.	sourceDeckId? / sourceUserId? (ficam null se a origem some). DeckSnapshotItem.card usa onDelete: Restrict — carta em lista histórica não pode ser apagada.
Post	Conteúdo editorial (notícia/preview/review/guia), Markdown.	postType , status , slug único.
SimulatorMatch	Persistência da partida do simulador (write-through do matchStore em memória — ver §7). Blob de estado autocontido, sem relação FK .	id (= matchId), state: Json (o GameState real, server-side, nunca vai redigido ao cliente), seats , deckKeys , version , phase , turnDeadlineAt: BigInt? , lastSeenAt: Json , gameOver: Json? , finishedAt? . @@index([updatedAt]) , @@index([finishedAt]) .

Não existe SimulatorBugReport — o reporte de bug do simulador (POST /api/simulator/matches/:id/report → reportSituation()) apenas loga o GameState + histórico no console do servidor com um reportId curto; não grava tabela.

3.2 Diagrama textual das relações

Sem FK no banco — as setas são relações a nível de aplicação (Prisma client + validação na API).

```

Season 1—* CardSet 1—* Card *—1 CardModel *—* CardRelation (self, curada)
    |
    |   * Ruling
    |   *—1 CardBanGroup (opcional)
    |
    * CardBinderItem *—1 CardBinder *—1 User
    * DeckItem *—1 Deck *—1 User
    * DeckSnapshotItem *—1 DeckSnapshot

User 1—* Deck 1—* DeckItem
User 1—* CardBinder 1—* CardBinderItem
User 1—* Post
User 1—* HostedEvent (como hoster)

Tournament 1—* TournamentEntry *—0..1 Deck
    *—0..1 User
    *—0..1 DeckSnapshot
    1—* TournamentEntryDeckChangeLog

HostedEvent 1—* HostedEventParticipant *—0..1 Deck / DeckSnapshot
    1—* HostedEventRound 1—* HostedEventMatch *—2 HostedEventParticipant (A / B?)

DeckSnapshot 0..1— Deck (sourceDeckId, SetNull)    0..1— User (sourceUserId, SetNull)

SimulatorMatch    (ilhado - nenhuma relação; id = matchId do matchStore)

```

4. Catálogo de cartas

Fonte: `scripts/catalog-import.mjs`, `prisma/seed-apitcg.mjs`, `prisma/*.mjs` de curadoria; `data/`; `docs/05`, `docs/09`, `docs/10`, `docs/13`.

4.1 Origem dos dados

Arquivo	Conteúdo
<code>data/apitcg-gundam.json</code>	Dataset amplo (APITCG) — ~1.812 impressões em 22 sets, com imagens, raridade, código de produto, stats. É a base do catálogo importado por <code>prisma:seed:apitcg</code> .
<code>data/gcg-official-cards.json</code>	Dados oficiais curados carta a carta (site oficial): texto de efeito EN, <code>textSectionsJson</code> (seções por gatilho), <code>traits</code> , <code>link</code> / <code>linkRefs</code> , <code>série</code> / <code>sourceTitle</code> . Fonte da curadoria e do motor do simulador (fixtures).
<code>data/catalog/</code>	Diretório opcional pra import local (<code>catalog:import:local</code>).

4.2 Pipeline de import / curadoria

Ordem (encadeada em `catalog:bootstrap`, ver §2.3):

1. `prisma:seed` — admin + registros de exemplo.
2. `prisma:seed:apitcg` (`prisma/seed-apitcg.mjs`) — cria `CardSet` e `Card` a partir do dataset APITCG. Normaliza URLs de imagem e `CardType` / `SetKind`.
3. `cardmodel:migrate:apply` (`prisma/migrate-card-model-data.mjs`) — para cada `code`, cria/atualiza um `CardModel` (identidade de jogo) e liga todas as impressões daquele code a ele. Ver `docs/13-migracao-cardmodel.md`. `cardmodel:validate` confere.
4. `curation:gcg:apply` (`prisma/apply-gcg-official-curation.mjs`) — cruza com `gcg-official-cards.json`: preenche `série`, `sourceTitle`, `textSectionsJson`, `CardRelation` (piloto de / apoia / arquétipo). Modo `--dry-run` por padrão, `--apply` grava.
5. `keywords:extract:apply` (`prisma/extract-keyword-effects.mjs`) — roda o parser `src/lib/gundam-card-effects.ts` sobre o texto e grava `triggerKeywords`, `effectKeywords`, `keywordTags` (com valor numérico, ex. `Repair 2`), `hasBurst` / `hasMain` / `hasAction` / `oncePerTurn`.
6. `banlist:apply` (`prisma/apply-official-banlist.mjs`) — `legalityStatus` por `CardModel` + `CardBanGroup` (par banido e regra de categoria genérica). Ver `docs/14`.
7. `rulings:import[:apply]` (`prisma/import-rulings.mjs`) — importa rulings oficiais traduzidos (fora da cadeia principal; ~90 rulings na v1.0).
8. Limpeza: `catalog:cleanup:phantom-prints[:apply]` remove impressões órfãs; `curation` e `banlist` têm sempre um modo `dry-run` antes do `--apply`.

Import também disponível pela API (admin): `POST /api/import/catalog`, `/api/import/cards`, `/api/import/rulings`, `/api/import/images-manifest`. `scripts/catalog-import.mjs` é a versão CLI (`catalog:import`), que reconstrói o texto de efeito a partir de `textSectionsJson` (labels em `[bracket reto]` — ver §5.4).

4.3 CardModel vs Card (prints)

Decisão de `docs/13`: **separar identidade de jogo de tiragem física**.

- `CardModel` — um por `code` de jogo. É o que o deckbuilder valida (cópias, cores, banlist via `banGroupId`), o que o motor do simulador consome e onde vivem `effectEn` / `effectPt`.
- `Card` — uma por impressão (reprint, promo, variante de arte, alt-rarity). Guarda `rarity`, `setId`, `imageUrl`, `printLabel`, `isPrimaryPrint`. `DeckItem` / `DeckSnapshotItem` / `CardBinderItem` apontam pra `Card` (uma impressão concreta), mas as regras "sobem" pro `CardModel` via `cardModelId`.

Rota de leitura do catálogo: `GET /api/cards` (filtros: cor, custo, tipo, trait, keyword, raridade, set, busca), `GET /api/cards/filters` (opções dos filtros), `GET /api/cards/:id` (detalhe, com `CardModel` + impressões + rulings), `GET /api/cards/:id/relations`, `GET /api/cards/:id/stats`.

4.4 Imagens

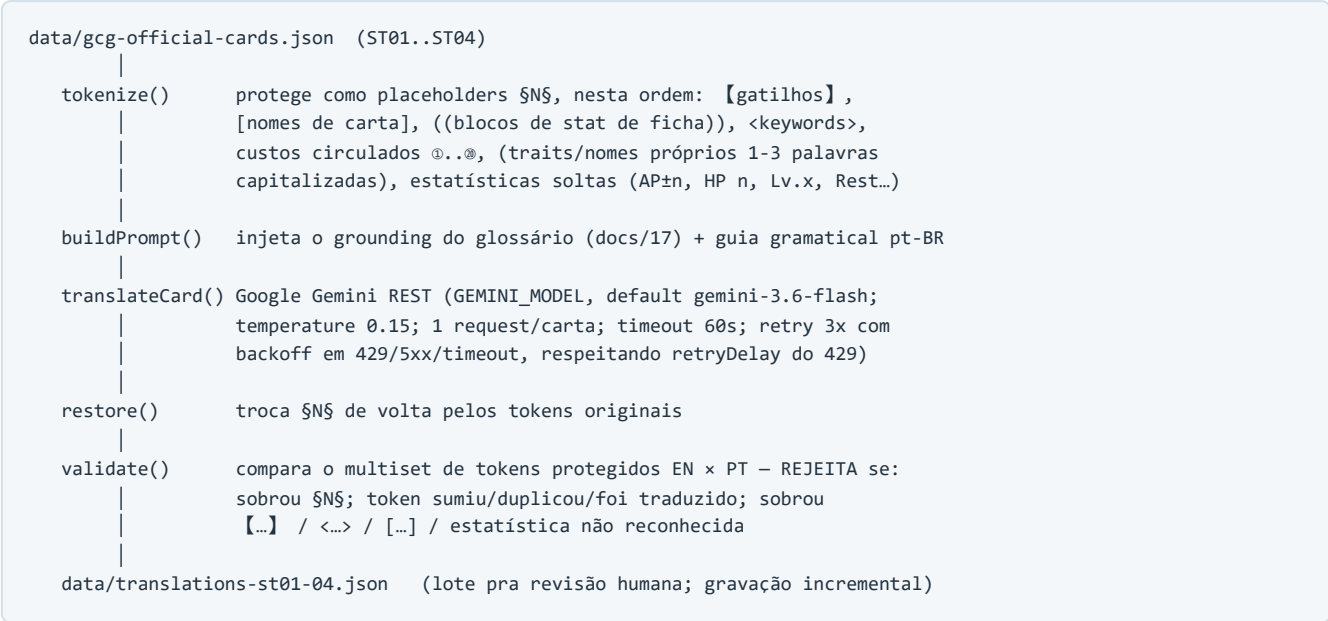
Cada `Card` tem várias URLs (`imageSmallUrl` ... `imageLargeUrl` , `thumbUrl` , `imageSourceUrl`). Storage por driver (`STORAGE_DRIVER`): `local` serve de `/uploads` pela própria API; `supabase` envia pro bucket (`SUPABASE_STORAGE_*`) com service role no backend. Há um `GET /api/image-proxy` pra contornar CORS/hotlink de fontes externas. Ver `docs/05-image-strategy.md` , `docs/09-storage-imagens.md` .

5. Tradução pt-BR do texto de efeito

Fonte: `scripts/translate-card-effects.mjs` (+ `.test.mjs`); `docs/17-glossario-traducao.md`, `docs/40`, `docs/42`.

Pipeline Node ESM puro que traduz o texto de efeito das cartas de ST01–ST04 (filtro `/^ST0[1234]-/`, 64 cartas; 52 com efeito real). **Regra de vocabulário** (`docs/17`): a keyword em si *nunca* traduz (`Deploy`, `Burst`, `Repair`, `<Blocker>` ...), assim como nomes de carta e atributos (AP/HP/Lv.); só a *explicação mecânica* vira português imperativo ("Compre 1 carta", "Escolha 1 Unidade inimiga", "Cause X de dano a ela").

5.1 Fluxo por carta



5.2 Tokenizer léxico e validador de integridade

O tokenizer é a peça central: ele garante que a tradução automática nunca corrompa um termo de jogo. Cada token protegido vira `§N§`; o LLM traduz só o texto ao redor. O **validador** roda depois do `restore()` e compara o *multiset* (conjunto com repetição) de tokens do EN contra o do PT — se algo não bate, a carta recebe `status: "REJEITADO"` com motivo, e não é aplicada. Isso pega tanto tradução indevida de keyword (`Implantar` , `<Bloqueador>`) quanto placeholder vazado ou token perdido.

5.3 Flags

Flag	Efeito
(nenhuma)	Roda o lote real (chama Gemini) e grava o JSON.
<code>--dry-run</code>	Tokenizer + <code>restore</code> + validador com tradução identidade (não chama API). Testa só o encanamento.
<code>--resume</code>	Mantém as cartas já <code>OK</code> no JSON, re-traduz só as pendentes/rejeitadas. Retoma quando a quota do free tier reseta.
<code>--revalidate</code>	Re-roda só o validador de tokens sobre o <code>effectPt</code> atual de cada linha (pra quando as traduções foram editadas à mão) e reescreve <code>status</code> / <code>motivo</code> .
<code>--apply</code>	Lê o JSON e imprime os UPDATE SQL das traduções OK (não conecta no banco). <i>Cuidado:</i> <code>--apply psql</code> no PowerShell corrompe UTF-8 (<code>【</code>) · e acentos viram <code>"?"</code>). Use <code>--apply > x.sql</code> e rode o arquivo, ou prefira <code>--push</code> .
<code>--push</code>	Aplica as traduções OK direto no Postgres via Prisma Client (<code>DATABASE_URL</code> do <code>.env</code>). Sem shell/pipe → UTF-8 intacto. Modo recomendado pra aplicar localmente.

Env: `GEMINI_MODEL`, `TRANSLATE_DELAY_MS` (pausa entre cartas, default 500 — subir pra ~15000 no free tier). Gravação é incremental (reescreve o JSON a cada carta); aborta sozinho após 3 falhas seguidas de quota. Chave em `.spartan/ai.env` (`GEMINI_API_KEY`).

5.4 Convenção de brackets

Contexto	Gatilho de efeito	Exemplo
JSON oficial (<code>gcg-official-cards.json</code>), fixtures e simulador (inspetor, <code>EffectSpec.trigger</code> , <code>textSectionsJson</code>)	<code>【X】</code> (bracket lenticular)	<code>【Deploy】</code> , <code>【Attack】</code> , <code>【Activate·Main】</code> , <code>【When Paired】</code> , <code>【Destroyed】</code> , <code>【Burst】</code>
Catálogo do portal (texto reconstruído por <code>catalog-import.mjs</code> → <code>buildEffectText</code> , e o parser <code>gundam-card-effects.ts</code>)	<code>[X]</code> (bracket reto)	<code>[Deploy]</code> , <code>[Burst]</code> — labels de seção no formato <code>[label]</code>

Ou seja: o mesmo gatilho aparece como `【Deploy】` no pipeline de dados/motor e como `[Deploy]` na renderização do catálogo. Nomes de carta usam `[Colchete]` nos dois; keywords de habilidade usam `<Chevron>` nos dois. Ao mexer no tokenizer ou no parser, manter os dois lados coerentes.

6. Motor do simulador

Fonte: `src/modules/simulator/engine/` + `content/`; `docs/18`, `docs/21`, `docs/29`, `docs/41`, `docs/45`.

O motor é **puro** — TypeScript sem dependência de React, rede ou Prisma. Roda igual em teste (`vitest`), no servidor autoritativo (`server/matchStore.ts`) e no MCP. É o núcleo do produto "simulador"; ver `docs/16` ("não é uma feature, é um segundo produto").

6.1 GameState serializável

Definido em `engine/types.ts`. 100% serializável (JSON) — é o que grava em `SimulatorMatch.state` e o que o MCP/busca clona. 9 zonas por jogador:

Zona	Regra
deck	50 cartas. Deck-out = derrota ao precisar comprar sem carta.
resourceDeck	10 cartas, separado. 1 compra por turno na Resource Phase.
shields	6 do topo do deck, face-down no setup. Cada shield = 1 HP; ao ser destruído, revela e checa 【Burst】 antes do trash.
resourceArea	máx. 15 recursos, máx. 5 EX Resource; 2º jogador começa com 1 EX Resource.
battleArea	máx. 6 Units; 1 Pilot pareia com 1 Unit.
baseSection	máx. 1 Base; EX Base deployada no setup.
trash	descarte (público).
exile	fora do jogo (<code>REMOVE_CARD_FROM_GAME</code> — EX Resource gasto etc.). Público, como o trash.
hand	máx. 10 — excesso descartado na End Phase.

Cada carta em jogo é uma `CardInstance` (`instanceId` = "A-12", `def`: `CardDef`, `owner`, `zone`, `rested`, `damage`, `statModifiers[]`, `keywordGrants[]`, `usedKeywordsThisTurn[]`, `enteredZoneOnTurn`, `pairedPilotId?` / `pairedUnitId?`, `cannotAttackUntilTurn?`). O `CardDef` é o dado estático achatado do `CardModel` (só os campos que o motor usa) — nos fixtures ST01–ST04 é dado literal copiado de `gcg-official-cards.json`, nenhum parser novo.

6.2 Reducer de eventos (`applyEvent` / `applyEvents`)

Todo efeito — keyword automática ou bespoke — é expresso como lista de `GameEvent` e aplicado por um reducer **puro** (`engine/events.ts`) que *nunca* muta o estado recebido (`cloneState` internamente). Isso torna o teste "comparar eventos gerados, não estado mutável" e dá base pronta pra log/replay auditável.

`GameEvent` (união discriminada por `type`) — os principais:

```
DRAW_CARD MOVE_CARD REST_CARD SET_ACTIVE PAIR_CARDS SPAWN_TOKEN
DAMAGE_UNIT DAMAGE_SHIELD DAMAGE_BASE HEAL_UNIT DESTROY_CARD REMOVE_CARD_FROM_GAME
MODIFY_STAT GRANT_KEYWORD MARK_KEYWORD_USED CLEAR_TURN_MODIFIERS MOVE_WITHIN_DECK
GRANT_ATTACK_TARGET_RELAX SET_CANNOT_ATTACK SET_SHIELD_PROTECTION SET_UNIT_DAMAGE_PROTECTION
PHASE_CHANGE TURN_CHANGE COMBAT_STEP_CHANGE ATTACK_DECLARED BLOCK_DECLARED
ACTION_PASS BEGIN/END_END_PHASE_ACTION_STEP END_PHASE_ACTION_PASS COMBAT_ENDED
SET_PENDING_DECISION CLEAR_PENDING_DECISION DISCARD_TO_HAND_LIMIT GAME_OVER
```

6.3 Estrutura de turno

5 fases (`engine/phases.ts`): **Start** (rested→active do ativo) → **Draw** (1) → **Resource** (1 do resourceDeck; o motor faz automático, não há ponto de decisão) → **Main** (jogar carta, **【Activate·Main】**, `<Support>`, atacar) → **End** (Action Step de fim de turno + descarte pra 10 + `<Repair>` + troca de turno).

O **Action Step** também acontece na **End Phase**, não só em combate. O motor nunca assume "é meu turno, só eu ajo" — há janelas explícitas em que o jogador em espera age.

6.4 Combate (engine/combat.ts , 5 etapas)

1. **Attack Step** — `declareAttack(state, attackerId, target)` . Valida: só Unit active do ativo; não recém-deployada (CR 3-2-4) salvo Link Unit (3-2-6-3); `attackTargetRules` (Zowort não mira o jogador; Wing Gundam pode mirar Unit active $\text{Lv.} \leq N$). Dispara `【Attack】` .
2. **Block Step** — `activateBlocker(state, blockerId)` / `skipBlock(state)` . `<Blocker>` muda o alvo; `<High-Maneuver>` do atacante desabilita.
3. **Action Step** — `passAction(state, player)` , alternado começando pelo jogador em espera, até dois passes consecutivos. Command `【Action】` e `【Activate·Action】` aqui.
4. **Damage Step** — `resolveDamageStep(state)` . Dano simultâneo = AP; `<First Strike>` antecipa; Base recebe no lugar dos shields; `<Breach N>` e `<Suppression>` (2 shields) calculados aqui; `combatTriggers` ("when this Unit's attack destroys...") e `pairedPilotFollowEvents` aqui.
5. **Battle End Step** — `resolveBattleEndStep(state)` . Efeitos "durante esta batalha" expiram (`shieldProtection` , `unitDamageProtection`).

8 keywords oficiais — nunca precisam de autoria manual (o dado vem estruturado do `CardModel`): `<Blocker>` , `<First Strike>` , `<High-Maneuver>` , `<Breach N>` , `<Suppression>` (em `combat.ts`); `<Support N>` , `<Repair N>` , `【Once per Turn】` (em `keywords.ts`).

Gatilho `【Destroyed】` — despachado por dois caminhos sem sobreposição (`docs/44` + `docs/45`): (a) *no* Damage Step (morte de batalha / Breach letal / combatTrigger letal): `actions.ts` chama `collectDestroyedInBattle` + `dispatchDestroyedTriggers` ; (b) *fora* do Damage Step (Unit morta por dano de efeito — Close Combat `【Main】` , Rewloola `【Deploy】`): `dispatcher.ts` → `dispatchDestroyedFromEffect` acha as Units que o efeito matou e dispara o `【Destroyed】` de cada uma. Não-pausante resolve inline; pausante (Char's Zaku II) vira `PendingDecision.abilityResolution` . Guarda anti-loop `MAX_DESTROYED_CHAIN = 8` ; cascata cross-player em fila FIFO.

6.5 DSL de efeitos: `EffectSpec` → `PrimitiveCall` → `GameEvent`

Camada 3 formalizada em `engine/effectSpec.ts` . Em vez de uma closure JS opaca, o texto bespoke de cada carta vira uma estrutura declarativa revisável lado a lado com o `effectEn` oficial:

```
interface EffectSpec {
  id: string;           // "<code>-<trigger>", ex. "ST01-010-Burst"
  cardCode: string;
  trigger: string;      // rótulo do textSectionsJson: "Deploy" | "Attack" |
                        // "Destroyed" | "Burst" | "Activate·Main" | ...
  cost?: PrimitiveCall[];
  condition?: { predicate: string; then: PrimitiveCall[]; else?: PrimitiveCall[] };
  actions: PrimitiveCall[];
  sourceText: string;   // effectEn da seção — NUNCA effectPt
  optional?: boolean; targetScope?: ...; targetFilter?: ...;
}

resolveEffectSpec(spec, ctx, predicateResolver?, targetFilterResolver?)
  → roda cost → condition → actions, compilando cada PrimitiveCall via
  compilePrimitive() para 0+ GameEvent.
```

Primitivas (`PrimitiveCall.op`) hoje disponíveis:

```
draw discard discardNamed damageShield damageUnit destroy heal
moveZone moveWithinDeck modifyStat grantKeyword rest setActive
addShieldToHand payResourceCost spawnToken spawnTokenByOwnUnitCount
peekAndReorderDeck preventShieldDamage preventUnitBattleDamage
preventAttackThisTurn grantAttackTargetRelax
deployFromHandTriggered lookAtTopFilterReveal
```

TargetRef: `{kind:"self"}` , `{kind:"pairedUnit"}` , `{kind:"instance"}` , `{kind:"named"}` (alvo escolhido externamente, colocado em `ctx.targets`), `{kind:"group", group: TargetGroup}` (`allFriendlyLinkUnits` , `allEnemyUnits{maxLevel?}` — computado do estado, sem escolha). Toda primitiva que consome alvo passa por `resolveTargetIds` (0+ ids).

Efeitos **não** gatilho→ação viram campo estruturado de `CardDef : staticAbilities[]` (During Pair / During Link contínuo, reavaliado a cada `effectiveAp / effectiveHp`), `combatTriggers[]` (During Link reage a evento de combate no Damage Step), `attackTargetRules` (legalidade de alvo), `link` (`satisfiesLinkCondition`).

6.6 PredicateResolver e TargetFilterResolver

`content/predicates.ts` — `defaultPredicateResolver`, `defaultTargetFilterResolver`.

Resolvers plugáveis, usados *tanto* pelos testes *quanto* pelo servidor real (uma implementação só). **Predicados**

(`EffectSpec.condition.predicate`): `pairedPilotHasTrait:<t>`, `pairedPilotLevelAtLeast:<n>`, `cardInTrashNamed:<name>`, `namedChoiceEquals:<key>:<val>`, `selfApAtLeast:<n>`, `controllerHasOtherLinkUnit`, `sourcePairedUnitIsLinkUnit`, `noControllerUnitTokenWithTrait:<t>`. **Filtros de alvo** (`targetFilter`): `hp<=N`, `level<=N`, `level>=N`, `ap<=N`, `hasKeyword:X`, `rested` — resolvidos por candidato em `computeLegalTargets` (escopo `enemyUnit / friendlyUnit / ownResource`), calculados uma vez no servidor. Novo padrão = novo membro da union, não reescrita do desenho.

6.7 dispatcher.ts / abilityDispatch.ts

- `dispatcher.dispatchTrigger(...)` — acha e resolve automaticamente todo `EffectSpec` de uma carta pra um trigger dado (Deploy / When Paired / Attack / Burst / Main / Action / Activate-Main), aplicando os eventos de cada spec antes do próximo, respeitando `【Once per Turn】` via `CardDef.oncePerTurn` + `usedKeywordsThisTurn`.
- `abilityDispatch.ts` — vocabulário "dispara agora, ou PAUSA pra o jogador resolver", compartilhado por `【When Paired】` e `【Attack】`. Se algum spec é `optional` ou precisa de alvo/escolha não fornecida, grava `PendingDecision.abilityResolution` e calcula *no servidor* o conjunto de opções legais (`legalTargets`, `handChoice.legalHandIds`, `deckTopReveal`) — `resolveAbility` valida a escolha do cliente contra ele, nunca confia cegamente.
- `dispatchBurstForNewlyTrashedShields()` — compara o estado antes/depois de um Damage Step, acha shields recém-trashadas com `hasBurst` + `EffectSpec` de Burst e oferece a ativação (por escolha de quem defende).

6.8 PendingDecision

Campo `GameState.pendingDecision: Record<PlayerId, PendingDecision | null>`. Enquanto um lado tem decisão pendente, **nenhuma ação de nenhum dos dois avança o estado** (`applyPlayerAction` recusa no topo). Variantes (por `kind`):

kind	Quando	Resolvido por
mulligan	Início da partida — manter ou trocar a mão de 5.	<code>resolveMulligan</code>
burst	Shield com <code>【Burst】</code> destruído no Damage Step (fila FIFO se várias).	<code>resolveBurstDecision</code> { <code>activate</code> , <code>targets?</code> }
abilityResolution	Efeito optativo / que pede alvo / escolha (ordem de gatilhos simultâneos, alvo, <code>handChoice</code> , <code>deckTopReveal</code>). Ex.: Full Frontal (deploy da mão), Char's Zaku II (revelar do topo).	<code>resolveAbility</code>
triggerOrder	2+ triggers de cartas diferentes no mesmo evento (tipo pronto; nenhuma carta ST01–04 dispara isso ainda).	<code>resolveTriggerOrder</code> { <code>orderedSpecIds</code> }
zoneOverflow	Zona acima do limite (mão > 10 na End Phase; futuro: 7ª Unit).	escolha de descarte

`handChoice` e `deckTopReveal` são *sub-estruturas* de `abilityResolution` (não `kind` próprio) — carregam os ids legais pré-computados no servidor.

6.9 Conteúdo por wave

`content/st0{1..4}.ts` — `EffectSpec` autorado carta a carta **contra o texto oficial EN** (nunca a tradução). `content/index.ts` agrega `ALL_EFFECT_SPECS`. Fixtures em `fixtures/st0{1..4}Deck.ts` (16 `CardDef` únicos + decklist 50+10 + tokens `T-001..T-005`) e `fixtures/vanillaDeck.ts` (deck sintético só-stats pra validar turno/combate).

- ST01 "Heroic Beginnings" / ST02 "Ruination Ablaze"** — cobertura 100% (as 8 lacunas de DSL originais foram fechadas: token, custo de recurso genérico, alvo em grupo, prevenção de dano condicional, peek-and-reorder, efeito estático During Pair/Link, `combatTrigger`, legalidade de alvo).

- **ST03 "Zeon's Fangs" / ST04 "Aile of Justice"** — as 32 cartas únicas jogáveis; novos padrões: revelar do topo (`lookAtTopFilterReveal`), deploy grátis da mão (`deployFromHandTriggered`), **【Destroyed】** dentro e fora de combate, concessão temporária de alvo, proibição de ataque no turno (`preventAttackThisTurn`), prevenção de dano condicional.

Cada carta nova segue o protocolo de `docs/29` : campo estruturado → primitiva → teste (TDD).

6.10 Redação de informação oculta

`engine/viewState.ts` — `viewStateFor(state, seat)` produz a `ViewGameState` que sai pra rede: mão do oponente e deck viram contagem; shields face-down não revelam identidade nem pro dono; trash e exile são públicos. O `GameState` completo **nunca** sai do servidor. `pendingDecision` é repassado aos dois lados só com o necessário (o `cardDef` do Burst já é público no trash).

7. Simulador — servidor autoritativo e rede

Fonte: `src/modules/simulator/server/matchStore.ts`, `socketBridge.ts`; `server/simulatorSocket.ts`; `src/modules/simulator/network/`; `server/index.ts`; `docs/23`, `docs/24`, `docs/39`.

7.1 `matchStore.ts` — motor 100% server-side

`Map<matchId, MatchRecord>` em memória como cache de trabalho. Cada `MatchRecord` guarda o `GameState` **real** — só `matchViewFor(match, seat)` / `viewStateFor` saem pra rede, um por jogador, já redigidos. As rotas HTTP e o Socket.io só chamam funções deste módulo; nenhuma toca `GameState` direto. API principal:

Função	Uso
<code>joinQueue</code> / <code>leaveQueue</code> / <code>queuePositionFor</code>	Fila de matchmaking automática (pareia 2 usuários diferentes). Fila efêmera — perder no restart é aceitável.
<code>createMatch</code> / <code>getMatch</code> / <code>joinMatch</code> / <code>seatFor</code>	Ciclo de vida da partida. <code>createMatch</code> já avança pro mulligan / Main Phase do turno 1.
<code>applyAction(matchId, userId, action)</code>	Borda ação→motor: autoriza (assento correto), chama <code>applyPlayerAction</code> (reducer que traduz cada <code>PlayerAction</code> pra função do motor), persiste, notifica assinantes.
<code>touchPresence</code> / <code>claimAbandonWin</code> / <code>resignMatch</code>	Presença (ping), W.O. por abandono, desistência.
<code>setAutoPass</code>	Liga o auto-pass do Action Step por assento.
<code>reportSituation</code>	Loga <code>GameState</code> + histórico no console com <code>reportId</code> curto (não grava tabela).
<code>subscribeAllMatches</code> / <code>subscribe(matchId)</code>	Assinatura pra broadcast (SSE e Socket.io se penduram aqui).
<code>loadMatch(matchId)</code>	Re-hidrata do Postgres sob demanda e re-armar o timer.
<code>decisionOwner</code> / <code>defaultActionFor</code>	De quem é a decisão agora, e qual ação o servidor toma se o timer estourar.

Timers

- `TURN_DECISION_MS = 300_000` (5 min) — decisão de Main Phase / decisão interativa (Burst, mulligan, ordem de gatilhos, resolução de habilidade).
- `ACTION_STEP_DECISION_MS = 30_000` — decisão de Action Step (só "Command **【Action】** ou passar").
- Estourou: `onTurnTimeout` aplica `defaultActionFor` (`skipBlock` / `passAction` / `finishTurn` conforme o passo).
- W.O. por abandono: após ~3 min sem sinal de vida do oponente, **nunca automático** — destrava um botão pro lado presente clicar (`claimAbandonWin`).
- Prioridade da "decisão atual": decisão interativa > combate > Action Step de fim de turno > Main Phase.

Persistência (`SimulatorMatch`)

Write-through *fire-and-forget*: o `Map` é o cache; a cada mutação, um `MatchPersistence` injetável (`setMatchPersistence`; `null` = no-op nos testes de motor) grava `StoredMatch` em `SimulatorMatch`. Assim a partida **sobrevive a restart / deploy / idle** do host. `finishedAt` separa vivas de terminadas; GC oportunista (partida terminada some ~10 min depois, na próxima escrita).

7.2 Transporte SSE (caminho padrão da v1.0)

Rotas em `server/index.ts` (todas `authRequired`, exceto o stream que aceita token na query):

```

POST /api/simulator/queue/join | /queue/leave      GET /api/simulator/queue/status
GET /api/simulator/matches/:id                  (snapshot redigido)
POST /api/simulator/matches/:id/actions          (PlayerAction)
POST /api/simulator/matches/:id/ping             (presença)
POST /api/simulator/matches/:id/report | /auto-pass | /claim-abandon-win | /resign
GET /api/simulator/matches/:id/stream            (Server-Sent Events; authFromQueryOrHeader)
# hoster-only (depuração, fora do fluxo normal):
GET/POST /api/simulator/matches POST /api/simulator/matches/:id/join

```

O stream SSE janelava o `eventLog` (últimos 150) por SSE em vez de reenviar o log inteiro; faz `unsubscribe` + `clearInterval` em `req.on("close")`.

7.3 Transporte Socket.io (`server/simulatorSocket.ts`)

Aditivo — roda AO LADO do SSE e das rotas POST, que ficam intactos. Pendura o `io` no mesmo HTTP server do Express.

- Path: `/api/simulator/socket`.
- Handshake: JWT em `auth.token` (mesmo `JWT_SECRET` / `jwt.verify` das rotas). Sem token → `guestId` efêmero assinado (TTL 12h) pra partida casual sem cadastro.
- Salas: `lobby:global` (todos), `match:{id}` e `match:{id}:{seat}` (isolamento por partida + por assento pra visão redigida).
- Broadcast de `match:view_update` sempre que o motor autoritativo muda — via `subscribeAllMatches` do `matchStore`, uma visão por jogador.
- Fila de matchmaking, convite direto (`challenge:*`) e ping/RTT.
- Contrato de eventos: `docs/39 §2.2`.

`src/modules/simulator/server/socketBridge.ts` — lógica **pura** de suporte (testável isolada, sem Node): `ChallengeRegistry` (convite direto) e `ActionDeduper` (dedup idempotente por `actionSeq`).

7.4 Cliente (`network/`)

- `socketClient.ts` — singleton Socket.io. Reconexão com backoff exponencial (500ms → 1s → 2s → 4s → teto 10s); ao (re)conectar reemite `match:join` e recebe o snapshot; fila de ações com `actionSeq` monotônico (idempotência); telemetria de ping; guarda o `guestToken` assinado do convidado sem login.
- `useMatchTransport.ts` — hook que a `SimulatorMatchPage` usa. **Socket.io é o transporte primário; fallback automático pro SSE** se o socket ficar `dead` (handshake recusado) ou não entregar o 1º snapshot em `SOCKET_FIRST_VIEW_TIMEOUT_MS` (6s — servidor sem Socket.io / proxy que bloqueia WS). Volta pro socket se ele se recuperar e reentregar uma view. Guarda de ordenação por `version` em `applyIncomingView`. Heartbeat de presença a cada 15s.

7.5 Convite por link ("Jogar com um amigo")

`ChallengeRegistry` (`socketBridge.ts`): o host gera um convite com código `GC-####` (prefixo `CHALLENGE_CODE_PREFIX = "GC"`). Cada host tem no máximo um convite vivo (criar de novo invalida o anterior); TTL `CHALLENGE_TTL_MS = 10 min`; `accept` consome o código (vira partida uma vez só). Eventos `challenge:created` (devolve o código) e `challenge:ready` (devolve o `matchId` quando o convidado aceita).

8. Deckbuilder e legalidade de deck

Fonte: `src/lib/deck-Legality.ts`, `src/lib/deck-Level-stats.ts`; `docs/14`, `docs/38`.

8.1 `deck-legality.ts` (Pacote A)

Lógica **pura** em `src/lib/` (não `server/`) de propósito — usada tanto pela API (`server/index.ts` importa daqui) quanto pelo deckbuilder no navegador (validação em tempo real, sem round-trip por clique).

Constantes: `DECK_MAIN_SIZE = 50`, `DECK_RESOURCE_SIZE = 10`, `DECK_MAX_COLORS = 2`, `DECK_MAX_COPIES_DEFAULT = 4`.

`computeDeckLegality(items, legality)` devolve uma lista de `DeckLegalityIssue` (não decide UX):

- **Tamanho** — `main` = exatamente 50; `resource` = exatamente 10. Seções `ex_base` / `ex_resource` (`NON_COUNTED_SECTIONS`) e `token_reference` não contam — são componente fixo do jogo.
- **Cores** — no máximo 2 cores distintas no deck principal.
- **Cópias** — até 4 por carta, salvo `restricted` (Map `CardModel.id` → cópias) ou `banned` (Set).
- **Ban groups** — `CardBanGroup`: no máximo `maxDistinct` escolhas diferentes do grupo por deck (cobre par banido e a regra de categoria genérica oficial). `DeckLegalityData.banGroups`.

`NON_STATS_SECTIONS` / `NON_STATS_CARD_TYPES` excluem recurso/EX/token de *toda* estatística de uso/metagame (senão apareceriam como "100% de presença").

8.2 `deck-level-stats.ts` (aba Estatísticas, `docs/38 §5`)

- **Curva de nível** — `buildLevelCurve(rows)` devolve sempre 6 faixas (Lv.1..5 e "6+", `LEVEL_CURVE_TOP_BUCKET = 6`), só Units com `level`. Cada barra é clicável no front (lista as Units da faixa).
- **Abertura sólida** — `hypergeometricAtLeastOne(populationSize, successCount, drawSize)`: $P(\geq 1 \text{ sucesso numa amostra sem reposição})$. Usado pra "chance de abrir com ≥ 1 Unit de nível baixo (Lv.1–3)" na mão de 5 (`OPENING_HAND_SIZE = 5`, `LOW_LEVEL_MAX = 3`) — mesma leitura hipergeométrica que o card "Mão inicial" já fazia pra custo baixo.

Persistência: `Deck` + `DeckItem` (`section` distingue `main` / `resource` / `ex_base` / `ex_resource` / `token_reference`). Capa/estilo visual: `Deck.coverImage` + `featuredCardIds[]` (bloco reposicionado na UI pra logo abaixo do nome/Salvar).

9. Ferramentas de desenvolvimento

Fonte: `src/modules/simulator/engine/legalActions.ts`, `selfPlay.ts`; `scripts/gundam-fuzz.mjs`; `scripts/mcp-gundam/.mcp.json`; `.github/workflows/ci.yml`; `docs/44`, `docs/46`.

9.1 Enumerador de ações legais (`engine/legalActions.ts`)

`enumerateLegalActions(state, seat, specs?, opts?)` → `LegalAction[]`, onde `LegalAction` é `PlayerAction` (1:1). Estratégia: gera candidatos amplos e **filtra** cada um por aplicação de teste (`applyPlayerAction` é puro). Erro de legalidade (`Error "cru"`) descarta o candidato; qualquer outro throwable (`TypeError` ...) é re-lançado — bug de motor de verdade. Cobre Main Phase (deploy Unit/Base/Pilot/Command, ataque, `Activate-Main`, `<Support>`, `finishTurn`), Block Step, Action Step (combate e fim de turno), e todas as `pendingDecision`.

Bug de motor conhecido (`docs/46` §Achados): `cloneState` não clona `endPhaseAction` a fundo. Workaround no módulo novo: `enumerateLegalActions` faz `structuredClone(state)` antes de cada aplicação de teste quando `state.endPhaseAction` está setado.

9.2 Self-play (`engine/selfPlay.ts`)

`runSelfPlay({ deckA, deckB, seed, policyA?, policyB?, maxTurns? })`. Policy `randomLegal` exportada. Detecta: exceção do motor (`crashed`), exceção ao enumerar, estado sem ação legal / travado (`illegalState`), invariante violada (`checkStateInvariants`), partida que não termina em `maxTurns`.

9.3 Fuzzing de regressão (`scripts/gundam-fuzz.mjs`)

```
corepack pnpm run gundam:fuzz -- --games=200 --seed=1 --maxTurns=200
corepack pnpm run gundam:fuzz -- --decks=ST01,ST03 --games=1 --seed=<seed> # repro de um achado
```

Roda N partidas `randomLegal` -vs- `randomLegal` por par de decks (todos os pares ST01–04 por padrão). Exit ≠ 0 se achar crash / estado ilegal / partida infinita, imprimindo `par + seed + ação` e a linha de repro. Rodada de Fase 1: 0 crashes / 0 estados ilegais / 0 partidas infinitas, média ~22 turnos.

9.4 Servidor MCP (`scripts/mcp-gundam/`)

Servidor MCP único que expõe o motor e o catálogo como ferramentas — base do bot de treino / fuzzing / MCTS (Fases 2–5). Registrado em `.mcp.json` (`{ "mcpServers": { "gundam": { "command": "node", "args": [ "scripts/mcp-gundam/server.mjs" ] } } }`); o Claude Code sobe o processo. `server.mjs` registra o loader `tsx/esm` antes de importar o motor TS — roda com `node` puro, sem build. Smoke: `corepack pnpm run mcp:gundam:smoke`.

HTTP (`mcp.mjs` → `attachMcpHttp(app, {...})`, `POST /mcp` Streamable HTTP) existe mas **não está plugado** no `server/index.ts` (decisão `docs/44`: o server é grande e não é type-checked).

Grupo	Tool	Uso
catalog	gundam_get_card	code → texto oficial EN + stats + CardDef do motor + EffectSpec[] + coverage + postgres (CardModel via Prisma, ou {available:false}).
	gundam_search_glossary	term → linhas de docs/17 que casam.
	gundam_coverage	set? → resumo por set (total / effectSpec / vanilla / faltando); com set , lista carta a carta.
sim (partidas efêmeras — Map + TTL 30min, nunca tocam SimulatorMatch)	sim_new	deckA , deckB (ST01..ST04), seed? → cria já na Main Phase do turno 1 (mulligan pulado). Devolve matchId .
	sim_view	matchId , seat → visão redigida.
	sim_legal	matchId , seat → enumerateLegalActions .
	sim_act	matchId , seat , action → aplica a PlayerAction , devolve novos eventos + visão.
	sim_selfplay	deckA , deckB , games? , seed? → agrega {crashes, illegalStates, unfinished, winA, winB, avgTurns} .

9.5 CI (.github/workflows/ci.yml)

Roda em PR pra dev e main , Node 22, corepack :

```
corepack pnpm install --frozen-lockfile
corepack pnpm exec prisma generate
corepack pnpm run check:types      # tsc -b
corepack pnpm test                  # vitest run
corepack pnpm run build              # tsc -b && vite build
corepack pnpm run gundam:fuzz -- --games=100  # regressão do motor, 100 partidas/par ST01-04
```

10. Convenções

Fonte: `.claude/rules/`, `docs/03`, `docs/10`, `AI_GUIDE.md`.

10.1 Nomenclatura e caso

Camada	Convenção
Banco (Postgres / Prisma <code>@map</code> quando preciso)	<code>snake_case</code>
Código TypeScript (frontend e backend)	<code>camelCase</code>
JSON na rede (payloads da API)	<code>snake_case</code>

Conversão nas bordas: `src/lib/api.ts` (axios) converte request `camelCase` → `snake_case` e response `snake_case` → `camelCase`. O backend serializa/desserializa em `snake_case`.

10.2 Tempo

Tudo **UTC**. Banco `DateTime` (Prisma) / `timestamp_tz`; código usa `Date` /ISO 8601 com `Z`; o frontend converte pra local só na exibição. `SimulatorMatch.turnDeadlineAt` é `BigInt` (epoch ms).

10.3 Banco

- **Sem foreign key / sem CASCADE** no catálogo e entidades de usuário — integridade na aplicação.
- **Soft delete** (`deletedAt`) nos modelos de catálogo/evento; toda query filtra `deletedAt: null`.
- `TEXT` em vez de `VARCHAR`; PKs `cuid()`.
- Migrations: `docs/11-checklist-migration.md` antes de mexer no schema.

10.4 Motor do simulador

- Motor **puro**, sem React/rede/Prisma. `GameState` 100% serializável.
- Reducers nunca mutam o estado recebido.
- `EffectSpec` autorado contra o `effectEn` **oficial**, nunca a tradução.
- Carta nova segue `docs/29`: campo estruturado de `CardDef` primeiro; primitiva de `EffectSpec` só se for gatilho→ação; TDD (teste contra o texto oficial da carta).
- Novo predicado / filtro / target group = novo membro da union em `predicates.ts` / `effectSpec.ts`, não reescrita.
- Regra de jogo confirmada contra Comprehensive Rules oficial antes de implementar (a Bandai atualiza sem aviso).

10.5 Testes

`corepack pnpm test` — cerca de 640 casos (`vitest run`). Convive com `check:types` (`tsc -b`) e `build` (`vite build`). `eslint` (`lint`, `lint:simulator`). CI roda os 4 + fuzz.

10.6 Git

- Commits: `<tipo><escopo>: <resumo>` — `feat`, `fix`, `refactor`, `style`, `docs`, `chore`, `test`. Escopos: `portal`, `cards`, `deckbuilder`, `simulator`, `prisma`, `docs` ...
- Branches: `feature/*` → `dev` → `main`. Tag por release (`chore(release): v1.0.0`).
- `dev` = staging; `main` = produção (Render).

11. Limitações conhecidas e backlog

Fonte: `CHANGELOG.md` [1.0.0], `docs/43`, `docs/45 §3`, `docs/46 §Achados`.

11.1 Rede / simulador (operacional)

- **Socket.io não promovido a produção** — o SSE segue como caminho padrão até a validação de rede com 2 jogadores reais em máquinas/redes diferentes. O cliente já tenta socket primeiro com fallback automático pro SSE.
- **Matchmaking ranqueado** — aceito no protocolo, sem fila própria nem ranking ainda.
- Fila de matchmaking é efêmera (perde no restart do servidor — aceitável; a *partida* persiste).
- Reporte de bug do simulador só loga no console (`reportId`), não grava tabela.

11.2 Motor — padrões GD/EB ainda não cobertos (docs/45 §3)

Nenhum é necessário pra ST01–ST04. Entram carta a carta quando as waves GD/EB forem autoradas (protocolo docs/29):

Padrão	Exemplo	O que falta
Dano a "1 to 2" alvos (multi-alvo GD/EB)	GD01-044 Kshatriya "Choose 1 to 2 enemy Units. Deal 1 damage."	<code>PendingDecision.abilityResolution</code> com contagem min/max; <code>damageUnit</code> iterar sobre ids.
<code>targetFilter</code> relativo à carta-fonte	GD01-093 Marida Cruz "Lv. equal to or lower than this Unit"	Passar <code>sourceInstanceId</code> no contexto do <code>TargetFilterResolver</code> + filtro <code>levelLteSource</code> .
【Destroyed】 direcionado que <i>pausa</i>	GD01-056 "【Destroyed】 Choose 1 enemy Unit with 5 or less AP. Deal 1 damage."	Quase pronto (<code>dispatchDestroyedFromEffect</code> já passa <code>targetFilterResolver</code>) — falta autorar o <code>EffectSpec</code> com <code>targetScope</code> / <code>targetFilter</code> e testar o pause com alvo.
Dano condicionado ao próprio estado (além de AP)	GD "if this Unit is (Zeon)", "while 5+ AP"	Novos predicados pontuais em <code>predicates.ts</code> (cor/trait da fonte, contagem de recursos...).
<Breach> condicional / concedido estaticamente	GD01-034 "【During Pair】 This Unit gains <Breach 3>"	Caminho estático contínuo pra keyword condicional (hoje só concessão por evento com <code>duration</code>).
Pilot pareado segue a Unit em morte por <i>efeito</i>	qualquer <code>damageUnit</code> / <code>destroy</code> letal numa Unit pareada	Gap pré-existente: <code>combat.ts</code> emite <code>pairedPilotFollowEvents</code> no Damage Step, <code>compilePrimitive</code> não (CR 3-3-6).

11.3 Motor — achados registrados

- `cloneState` não clona `endPhaseAction` a fundo (só spread) — corromper o snapshot original ao aplicar 2 ações do MESMO snapshot no Action Step da End Phase. Inócuo no `matchStore` (descarta o state antigo a cada ação); quebra consumidores que especulam (MCTS, validação do enumerador). Workaround: `structuredClone` no `legalActions.ts`. Correção de verdade: clonar `endPhaseAction` como já se faz com `combat`.
- Confirmado **não** ser bug: dano-por-efeito (`damageUnit` / `destroy` de `EffectSpec`) *nunca* dispara `<Breach>` / `<Suppression>` / `combatTrigger` — esses só rodam dentro do `resolveDamageStep`. Teste de regressão em `engine/destroyedOutOfCombat.test.ts`.

11.4 Produto (roadmap — CHANGELOG.md "No radar")

- Bot de treino (heurístico → IA) e pipeline de correção assistida (Fases 2–5 do simulador).
- Analytics competitivos mais profundos (meta por temporada/arquétipo, uso por carta) — Fase 2.
- Perfis públicos, decks favoritos/compartilháveis — Fase 3.
- Ranking no simulador — Fase 4.
- 【Pilot】 [X] como pré-requisito de *jogar* a carta (legalidade de jogo, nunca foi uma das 8 lacunas de DSL) — ponte deckbuilder → simulador.
- Cobertura de efeitos das coleções GD/EB (ver §11.2).